COMPILER DESIGN
Practicals

Name:- **Satyam Yadav**
Roll No:- **16072**
Course:- **BSc.(Hons.) Computer Science, Sec B**

Q1) Write a Lex program to count the number of lines and characters in the input file.

```
%{
#include <stdio.h>
int line_count = 0;
int char_count =0;
%}

%%
\n {line_count++; char_count++;}
. {char_count++;}

%%

int main(){
yylex();
printf("Number of lines: %d\n",line_count);
printf("Number of characters: %d\n", char_count);
return 0;
}

int yywrap(){
return 1;}
```

```
➜ CD project flex q.l
➜ CD project gcc lex.yy.c
➜ CD project ./a.out
hello , this is a dog
mitrr aise mt karo
hello ji
Number of lines: 3
Number of characters: 52
➜ CD project
```

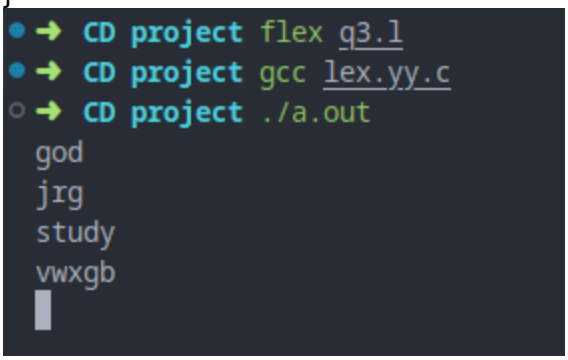
Q2) Lex program to count the number of vowels and consonants in a given string

```
%{
#include <stdio.h>
int vowels =0;
int consonants = 0;
%}

%%
[aeiouAEIOU] {vowels++;}
[a-zA-Z] {consonants++;}
.
%%

int main(){
printf("Enter string:");
yylex();
printf("Num of vowels: %d\n",vowels);
printf("Num of consonants: %d\n",consonants);
return 0;
}

int yywrap(){
return 1;
}
```

```
➜ CD project flex q2.l
➜ CD project gcc lex.yy.c
➜ CD project ./a.out
Enter string:aeioubossdog

Num of vowels: 7
Num of consonants: 5
➜ CD project
```

Q3) Write a Lex program that implements the Caesar cipher: it replaces every letter with the one three letters after in in alphabetical order, wrapping around at Z. e.g. a is replaced by d, b by e, and so on z by c.

```
%{
#include <stdio.h>
%}

%%

[a-z]  {
        /* shift lowercase letters */
        char c = yytext[0];
        c = ((c - 'a' + 3) % 26) + 'a';
        printf("%c", c);
      }

[A-Z]  {
        /* shift uppercase letters */
        char c = yytext[0];
        c = ((c - 'A' + 3) % 26) + 'A';
        printf("%c", c);
      }

.      { printf("%s", yytext); }   /* print any other character as-is */

%%
int yywrap(){
      return 1;
}

int main() {
    yylex();
    return 0;
}
```



Q4) Write a Lex program that finds the longest word (defined as a contiguous string of upper and lower case letters) in the input.

```
%{
#include <stdio.h>
#include <string.h>

char longest[1024] = "";
%}

%%

[A-Za-z]+   {
                if (strlen(yytext) > strlen(longest)) {
                    strcpy(longest, yytext);
                }
            }

.|\n        ;

%%

int yywrap() {
    return 1;
}

int main() {
    yylex();
    printf("Longest word: %s\n", longest);
    return 0;
}
```

```
➜ CD project flex q4.l
➜ CD project gcc lex.yy.c
➜ CD project ./a.out
a bc def ghij klmnopq
Longest word: klmnopq
➜ CD project
```

Q5) Write a Lex program that distinguishes keywords, integers, floats, identifiers, operators, and comments in any simple programming language.

```
%{
#include<stdio.h>
#include<string.h>

%}

%%
"int"|"float"|"if"|"else"|"while"|"for"|"return"|"void" {printf("keyword: %s\n", yytext);}
[A-Za-z_][A-Za-z0-9_]*  {printf("identifiers:%s/n", yytext);}
[0-9]+\.[0-9]+        {printf("Float:%s/n",yytext);}
[0-9]+ {printf("Integer:%s/n", yytext);}
"=="|"!="|"<="|">="|"="|"+"|"-"|"*"|"/"|"<"|">" {printf("Operator:%s\n",yytext);}
[ \t\n]+
.  {printf("unknown: %s\n", yytext);}

%%

int yywrap() {
    return 1;
}

int main(int argc, char *argv[]){
if(argc>1){
FILE *f = fopen(argv[1], "r");
if(!f){
printf("cannot open file %s\n", argv[1]);
return 1;
}
yyin = f;
}
yylex();
    return 0;
}
```

```
➜  CD project flex q5.l
➜  CD project gcc lex.yy.c
➜  CD project ./a.out
while for * + 64 dat
keyword: while
keyword: for
Operator:*
Operator:+
Integer:64/nidentifiers:dat/n
```

Q6) Write a Lex program to count the number of words, characters, blank spaces and lines in a C file.

```
%{
#include <stdio.h>
int word_count = 0;
int char_count = 0;
int space_count = 0;
int line_count = 0;
%}
%%
[A-Za-z0-9_]+       { word_count++; char_count += yyleng; }

" "                 { space_count++; char_count++; }

"\t"                { char_count++; }       /* tabs count as characters */
\n                  { line_count++; char_count++; }

.                   { char_count++; }       /* any other single character */
%%
int main() {
    yylex();
    printf("Number of words: %d\n", word_count);
    printf("Number of characters: %d\n", char_count);
    printf("Number of blank spaces: %d\n", space_count);
    printf("Number of lines: %d\n", line_count);
    return 0;
}

int yywrap(){
   return 1;
}
```

```
➜ CD project flex q6.l
➜ CD project gcc lex.yy.c
➜ CD project ./a.out
^[[C^[[C^[[C^Z
[7]  + 25051 suspended  ./a.out
➜ CD project ./a.out
sample.c
^Z
[8]  + 25169 suspended  ./a.out
➜ CD project ./a.out
a.exe < sample.c
Number of words: 4
Number of characters: 17
Number of blank spaces: 2
Number of lines: 1
➜ CD project
```

Q7) Write a Lex specification program that generates a C program which takes a string "abcd" and prints the following output: abcd abc ab a

```
%{
#include <stdio.h>
#include <string.h>
%}

%%

[a-zA-Z]+   {
                int n = strlen(yytext);
                for (int len = n; len >= 1; --len) {
                    for (int i = 0; i < len; i++)
                        putchar(yytext[i]);
                    putchar('\n');
                }
            }

[ \t\r\n]+  ;   /* ignore whitespace */
.           ;   /* ignore any other character */

%%
int main(int argc, char **argv)
{
    printf("Enter a string: ");
    yylex();
    return 0;
}
int yywrap() {
return 1;
 }
```

```
➜ CD project flex q7.l
➜ CD project gcc lex.yy.c
➜ CD project ./a.out
Enter a string: abcd
abcd
abc
ab
a
saty
saty
sat
sa
s
```

Q8) A program in Lex to recognise a valid arithmetic expression.

```
%{
#include<stdio.h>
%}


%%
[0-9]+ {printf("Number: %s\n",yytext);}
"+"|"-"|"*"|"/" {printf("Operator:%s\n",yytext);}
"(" {printf("left paranthesis:%s\n",yytext);}
")" {printf("right paranthesis:%s\n",yytext);}
[\t]+;
.    {fprintf(stderr,"Error:Invalid  Character'%s'\n",yytext);}
%%

int main(){
yylex();
return 0;
}

int yywrap(){
return 1;
}
```

```
➜ CD project flex q8.l
➜ CD project gcc lex.yy.c
➜ CD project ./a.out
()^[[D^Z
[6]  + 16383 suspended  ./a.out
➜ CD project ./a.out
(45+0)*2
left paranthesis:(
Number: 45
Operator:+
Number: 0
right paranthesis:)
Operator:*
Number: 2
```

Q9(a) Write a YACC program to find the validity of a given expression (for operators + - * and /)

Lex file

```
%{
#include "y.tab.h"
#include <stdio.h>
%}

%%

[0-9]+      { return NUMBER; }
[ \t\n]+    ;              /* ignore whitespace */
"+"         { return PLUS; }
"-"         { return MINUS; }
"*"         { return MUL; }
"/"         { return DIV; }
"("         { return LP; }
")"         { return RP; }

.           { return yytext[0]; }
```

```
%%

int yywrap() {
    return 1;
}
```

Yacc file

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

%token NUMBER
%token PLUS MINUS MUL DIV
%token LP RP

%%

start: expr        { printf("Valid Expression\n"); }
     ;

expr: expr PLUS term
    | expr MINUS term
    | term
    ;

term: term MUL factor
    | term DIV factor
    | factor
    ;

factor: NUMBER
      | LP expr RP
      ;

%%

int main() {
    printf("Enter expression: ");
```

```
    yyparse();
    return 0;
}

void yyerror(const char *s) {
    printf("Invalid Expression\n");
}
```

```
➜ CD project flex q9.l
➜ CD project yacc -dy q9.y
➜ CD project gcc lex.yy.c y.tab.c
➜ CD project ./a.out
Enter expression: (9+5*5/8-2)
Valid Expression
➜ CD project
```

Q9(b) A program in YACC which recognises a valid variable which starts with letter followed by a digit. The letter should be in lowercase only.

Lex file:-

```
%{
#include "y.tab.h"
#include <stdio.h>
%}

%%

[a-z][0-9]      { return VALID_VAR; }

[ \t\n]+        ;   /* ignore whitespace */

.               { return INVALID; }   /* anything else is invalid */

%%

int yywrap() {
    return 1;
}
```

Yacc file:-

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

%token VALID_VAR INVALID

%%

start:
        VALID_VAR   { printf("Valid variable\n"); }
      | INVALID     { printf("Invalid variable\n"); }
      ;

%%

int main() {
    printf("Enter a variable: ");
    yyparse();
    return 0;
}

void yyerror(const char *s) {
    printf("Invalid variable\n");
}
```

```
➜ CD project flex q9.l
➜ CD project yacc -dy q9.y
➜ CD project gcc lex.yy.c y.tab.c
➜ CD project ./a.out
Enter a variable: t12
Valid variable
Invalid variable
➜ CD project ./a.out
Enter a variable: sat
Invalid variable
Invalid variable
```

Q10) A Program in YACC to evaluate an expression (simple calculator program for addition and subtraction, multiplication, division).
Lex file:-

```
%{
#include "y.tab.h"
#include <stdio.h>
%}

%%

[0-9]+          { yylval = atoi(yytext); return NUMBER; }

"+"             { return PLUS; }
"-"             { return MINUS; }
"*"             { return MUL; }
"/"             { return DIV; }
"("             { return LP; }
")"             { return RP; }

[ \t\n]+        ;   /* ignore whitespace */

.               { return yytext[0]; }

%%

int yywrap() {
    return 1;
}
```

Yacc file:-

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

%token NUMBER
%token PLUS MINUS MUL DIV LP RP
```

```
%%

start: expr {
                printf("Result = %d\n", $1);
            }
      ;

expr: expr PLUS term        { $$ = $1 + $3; }
    | expr MINUS term       { $$ = $1 - $3; }
    | term
    ;

term: term MUL factor       { $$ = $1 * $3; }
    | term DIV factor       {
                                if ($3 == 0) {
                                    printf("Error: Division by zero\n");
                                    exit(1);
                                }
                                $$ = $1 / $3;
                            }
    | factor
    ;

factor: NUMBER
      | LP expr RP          { $$ = $2; }
      ;

%%

int main() {
    printf("Enter expression: ");
    yyparse();
    return 0;
}

void yyerror(const char *s) {
    printf("Invalid expression\n");
}
```

```
➜ CD project flex q10.l
➜ CD project yacc -dy q10.y
➜ CD project gcc lex.yy.c y.tab.c
➜ CD project ./a.out
Enter expression: 2+4*8
Result = 34
➜ CD project
```

Q11) Program in YACC to recognize the string „aaabbb", „aabb" „ab" of the langauge (an b n , n>=1).

```
%{
#include "y.tab.h"
%}

%%

a       { return A; }
b       { return B; }
\n      { return '\n'; }
[ \t]   ;
.       { return INVALID; }

%%

int yywrap()
{ return 1;
 }
```

YACC file :

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);
%}

%token A B INVALID

%%
```

```
start:
      S '\n'     { printf("Valid string\n"); }
    | '\n'       { yyerror("Empty input"); }
    ;

S:
      A S B
    | A B
    ;

%%

void yyerror(const char *s) {
    printf("Invalid string\n");
}

int main() {
    printf("Enter string: ");
    yyparse();
    return 0;
}
```

```
➜ CD project flex q11.l
➜ CD project yacc -dy q11.y
➜ CD project gcc lex.yy.c y.tab.c
➜ CD project ./a.out
Enter string: abb
Invalid string
➜ CD project ./a.out
Enter string: aabb
Valid string
```

Q12) Program in YACC to recognise the language (an b, n>=10). (output to say input is valid or not)

```
%{
#include "y.tab.h"
#include <stdio.h>
%}

%%
```

```
a      { return A; }
b      { return B; }
[ \t\n] ;   /* ignore whitespace */
.      ;    /* ignore invalid characters */

%%

int yywrap() {
    return 1;
}
```

YACC file:-

```
%{
#include <stdio.h>
#include <stdlib.h>

int count = 0;

void yyerror(const char *s);
int yylex();
%}

%token A B

%%

start: S B  {
                if (count >= 10)
                    printf("Valid string \n");
                else
                    printf("Invalid string \n");
            }
     ;

S : A S   { count++; }
  | A     { count++; }
  ;

%%

int main() {
```

```
    printf("Enter string: ");
    yyparse();
    return 0;
}

void yyerror(const char *s) {
    printf("Invalid string\n");
}
```

```
➜ CD project flex q12.l
➜ CD project yacc -dy q12.y
➜ CD project gcc lex.yy.c y.tab.c
➜ CD project ./a.out
Enter string: aaaaaaaaaaaaaaaaaaaaaab
Valid string

aaaaab
Invalid string
➜ CD project
```